Module 3: Intractable Problems

Serge Gaspers (K17 506)

Course Admins: cs3821@unsw.edu.au
(Liam, Gerald, Song)

School of Computer Science and Engineering
UNSW Sydney

Term 3, 2025

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Polynomial Time Algorithms

Definition

An algorithm is polynomial time if for every input it terminates in polynomially many steps in the length of the input.

- This means that there is a natural number c (independent of the input) so that the algorithm terminates in $T(n) = O(n^c)$ steps, where n is the size of the input.
- This is sometimes written $n^{O(1)}$.
- Examples:
  - merge sort has worst-case running time $O(n \log n)$, and since $O(n \log n) \subseteq O(n^2)$ it runs in polynomial time, but
  - the brute force algorithm for longest increasing subsequence (for each of the 2^n subsequences of the input sequence, check whether it is increasing, and update the max length) takes at least $n\,2^n$ operations, so it is not $O(n^c)$ for any integer c.

Decision Problems and Class **P**

Definition

A decision problem is a problem with a YES or NO answer.

Examples include:

- "Given an input number x, is it a prime number?"
- "Given an input graph G, is it connected?"
- "Given an input graph G, does it have a cycle containing all vertices of G?"

Definition

A decision problem A is in the class **P** (polynomial time, denoted $A \in \mathbf{P}$) if there is a polynomial time algorithm for A.

Class **NP**

Definition

A decision problem A is in the class **NP** (non-deterministic polynomial time, denoted $A \in \mathbf{NP}$) if there is a decision problem $B \in \mathbf{P}$ such that

- for every input x, $A(x)$ is YES iff there is some y with $|y| \leq |x|^{O(1)}$ for which $B(x, y)$ is YES, and

- y is the certificate: it provides evidence for why x is a YES instance of problem A

- B is the certifier: it checks that evidence in polynomial time.

Example: Primality Testing

Question

Consider the decision problem $A =$ "Given an integer x, is x <u>not</u> prime?" Is $A \in \mathbf{NP}$?

Answer

- We need to find a problem B such that $A(x)$ is YES if and only if there is some y for which $B(x, y)$ is YES.
- The natural choice is $B(x, y) =$ "is x divisible by y?"
- $B(x, y)$ can indeed be solved by a polynomial time algorithm, and it is sufficient to consider candidate certificates $y \leq \sqrt{x}$.

Example: Primality Testing

Question

Is $A \in \mathbf{P}$?

Answer

- Also yes! But this is not at all straightforward. This is a famous and unexpected result, proved in 2002 by Agrawal, Kayal and Saxena.
- The AKS algorithm provides a <u>deterministic</u>, <u>polynomial time</u> procedure for testing whether an integer x is prime.

Example: Primality Testing

The length of the input for primality testing is $O(\log x)$.

Comparing some well-known algorithms for primality testing:

- The naïve algorithm tests all possible factors up to the square root, so it runs in $O(\sqrt{x})$ time and is therefore not a polynomial time algorithm.
- The Miller-Rabin algorithm runs in time proportional to $O(\log^3 x)$ but determines only <u>probable primes</u>.
- The original AKS algorithm runs in $\tilde{O}(\log^{12} x)$, and newer versions run in $\tilde{O}(\log^6 x)$.[1]
- However, the AKS algorithm is rarely used in practice; tests using elliptic curves are much faster.

[1] the $\tilde{O}$ notation hides polylogarithmic factors in the input size, so $\tilde{O}(\log^6 x)$ is actually $O(\log^6 x \cdot \log^c \log x)$ with $c \in O(1)$.

Example: Satisfiability

Satisfiability (SAT)

Input: a propositional formula in conjunctive normal form (CNF) $C_1 \wedge C_2 \wedge \ldots \wedge C_n$ where each clause C_i is a disjunction of propositional variables or their negations, for example

$$(x_1 \vee \neg x_2 \vee x_3 \vee \neg x_5) \wedge (x_2 \vee x_3 \vee \neg x_5 \vee \neg x_6) \wedge (\neg x_3 \vee \neg x_4 \vee x_5)$$

Question: Is there an assignment to the variables which makes the formula true?

- SAT is in **NP**: given an assignment to the variables one can determine in polynomial time whether the formula evaluates to true.

Example: Satisfiability

Question

Is SAT in **P**?

Answer

Unknown

- kSAT is the restriction of SAT to inputs where each clause has exactly k variables.
- 2SAT can be solved in linear time using strongly connected components and topological sort.
- We will see that 3SAT is as "hard" as SAT.

Partial Answer

2SAT $\in$ **P**.
3SAT $\in$ **P** $\iff$ SAT $\in$ **P**.

Question

Could it be that <u>every</u> problem in **NP** is also in **P**?

- For example, is there a polynomial time algorithm to solve SAT?
- If so, given a propositional formula with n variables, then solving SAT, i.e.,

  finding out whether any of the 2^n assignments of true/false to the variables satisfies the formula

  would be not much harder than verifying a certificate for SAT, i.e.,

  checking whether a particular assignment of true/false to the variables satisfies the formula.

P vs NP

- Your intuition may vary:
  - solving 2SAT is actually not much harder than verifying a solution to 2SAT, but
  - most people think that this does <u>not</u> apply to SAT.
- However, so far no one has been able to prove (or disprove) that there are problems in **NP** that cannot be solved in polynomial time, despite decades of effort by famous people!
- The conjecture that **NP** is a strictly larger class of decision problems than **P** is known as the "**P** $\neq$ **NP**" hypothesis, and it is widely considered to be one of the most important open problems in mathematics and computer science.

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Reductions

- You have already used the technique of problem solving by reduction.
- This is when we map a new problem back to an old problem that we already know how to solve. For example:
  - given a graph with coloured vertices and two vertices u, v, find a shortest $u - v$ path that avoids monochromatic edges can be reduced to the problem of finding a shortest $u - v$ path in a graph
  - finding an edge-clique cover of an interval graph can be reduced to the interval stabbing problem
- This gives an algorithm for the new problem, by applying the reduction algorithm, followed by the known algorithm for the old problem.

Notes

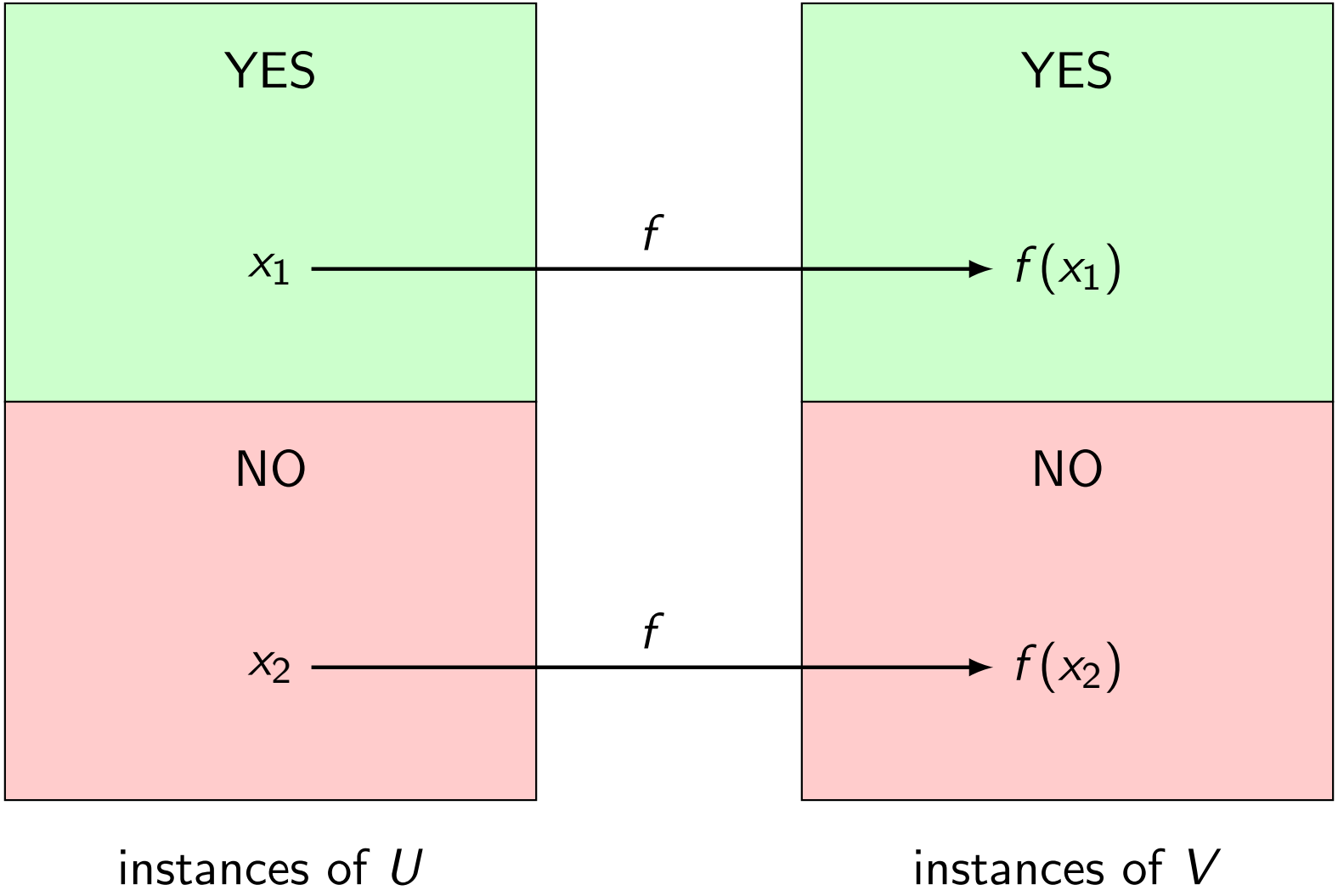
- A reduction from A to B implies that solving A is no more difficult than executing the reduction algorithm, followed by an algorithm for B.
- A simple reduction from kSAT, $k \in \{2, 3\}$ to SAT simply outputs the input instance.
- This shows that 2SAT and 3SAT are no more difficult to solve than SAT.

Definition (Karp reduction)

Let U and V be two decision problems.
We say that U is polynomially reducible to V if there is a function f such that:

1. f maps each instance x of U to an instance $f(x)$ of V.
2. f maps YES instances of U to YES instances of V and NO instances of U to NO instances of V, i.e., $U(x)$ is YES if and only if $V(f(x))$ is YES.
3. $f(x)$ can be computed in polynomial time.

This is sometimes denoted $U \leq_P V$.


YES
YES
x_1
f
$f(x_1)$
NO
NO
x_2
f
$f(x_2)$
instances of U
instances of V

Polynomial Reductions

- If there is a polynomial reduction from U to V, we can conclude that U is <u>no harder than</u> V (w.r.t. polynomial-time computability).
- If you could solve problem V in polynomial time, then problem U would also have a polynomial time algorithm.
  - For an instance x of U, you can instead solve the instance $f(x)$ of V, which has the same answer.
- The contrapositive is also true: if there is no (known) polynomial time algorithm for U, then there also can't be a (known) polynomial time algorithm for V.

Example: Reduction of SAT to 3SAT

Claim

Every instance of SAT is polynomially reducible to an instance of 3SAT.

Proof Outline

We introduce more propositional variables and replace every clause by a conjunction of several clauses.

For example, we replace the clause

$$\underbrace{x_1 \vee \neg x_2} \vee \underbrace{\neg x_3} \vee \underbrace{x_4} \vee \underbrace{\neg x_5 \vee x_6} \tag{1}$$

with the following conjunction of "chained" 3-clauses with new propositional variables Q_1, Q_2, Q_3:

$$\begin{aligned} &(\underbrace{x_1 \vee \neg x_2} \vee q_1) \wedge (\neg q_1 \vee \underbrace{\neg x_3} \vee q_2) \\ &\wedge (\neg q_2 \vee \underbrace{x_4} \vee q_3) \wedge (\neg q_3 \vee \underbrace{\neg x_5 \vee x_6}) \end{aligned} \tag{2}$$

Claim

For any assignment to the x_i variables that satisfies (1), there is an assignment to the q_j variables that also makes (2) true.
Every assignment which satisfies (2) also satisfies (1).

Clearly, (2) can be obtained from (1) using a simple polynomial time algorithm.

Cook's Theorem

Theorem

Every problem in **NP** is polynomially reducible to SAT.

- This means that for every decision problem U in **NP**, there is a polynomial time computable function f such that:
  1. for every instance x of U, $f(x)$ is a valid propositional formula for the SAT problem, and
  2. $U(x)$ is true iff the formula $f(x)$ is satisfiable.
- The proof encodes non-deterministic Turing machines as SAT formulas. It is beyond the scope of this course.

NP-hard

Definition

A decision problem V is NP-hard if every other problem in **NP** is polynomially reducible to V.

- Informal meaning: V is NP-hard if <u>it is at least as hard as every problem in **NP**</u>.
- Cook's Theorem says that SAT is NP-hard.

Common misconception

An NP-hard problem might not be in **NP**! The famous Halting Problem is an example.

Definition

A decision problem is NP-complete if it is in **NP** and NP-hard.

- We saw earlier that SAT is in **NP**, and from Cook's theorem it is also NP-hard, so it is NP-complete.
- NP-complete problems are in a sense universal. If we had a polynomial-time algorithm which solves an NP-complete problem V, then we could obtain a polynomial-time algorithm to solve every other problem U in **NP** by reduction:
  1. compute in polynomial time the reduction $f(x)$ from U to V,
  2. then run the algorithm that solves V on instance $f(x)$.
- A polynomial time algorithm for <u>any</u> NP-complete problem would imply that $\mathbf{P} = \mathbf{NP}$.

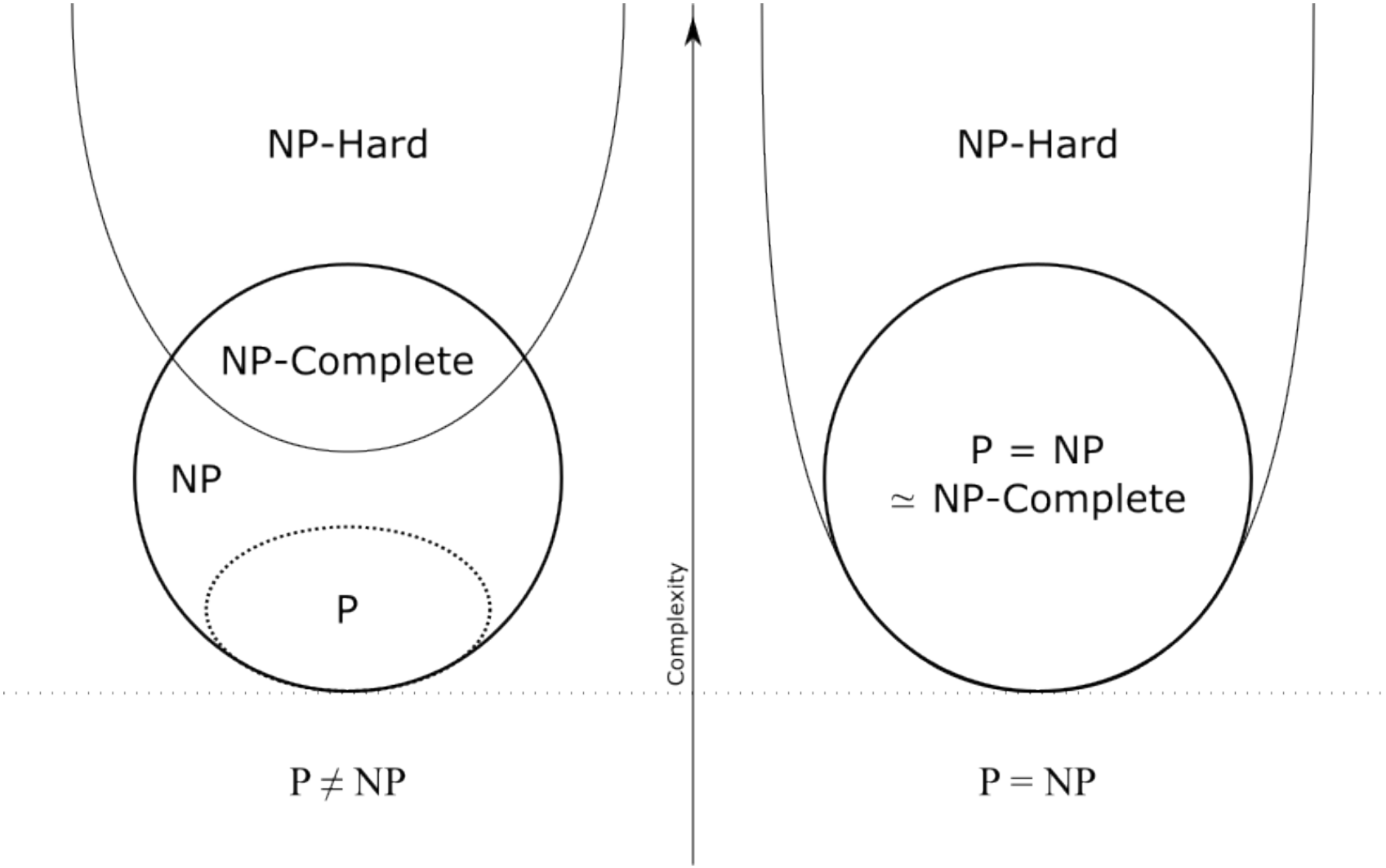

NP-Hard
NP-Complete
NP
P
Complexity
NP-Hard
P = NP
≃ NP-Complete
P ≠ NP
P = NP

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Why do we care about NP-complete problems?

- So NP-complete problems are the hardest problems in **NP** – a polynomial time algorithm for solving an NP-complete problem would make every other problem in **NP** also solvable in polynomial time.

- But if **P** $\neq$ **NP** (as is commonly hypothesised), then there cannot be any polynomial time algorithms for solving an NP-complete problem, not even an algorithm that runs in $O(n^{1,000,000})$ time.

Why do we care about NP-complete problems?

- Maybe SAT is not that important - why should we care about satisfiability of propositional formulas?
- Maybe NP-complete problems only have theoretical significance and no practical relevance?
- Unfortunately, this could not be further from the truth!
- A vast number of practically important decision problems are NP-complete!

Travelling Salesperson Problem

Instance:

1. a map, i.e., a weighted directed graph with:
   - vertices representing locations
   - edges representing roads between pairs of locations
   - edge weights representing the lengths of these roads;
2. a number ℓ.

Problem: Is there a tour along the edges which visits each location (i.e., vertex) exactly once and returns to the starting location, with total length at most ℓ?

The special case where the graph is undirected (i.e., for each directed edge in the graph, the reverse edge is also in the graph) and the edge weights are all equal to 1 is known as the Hamiltonian Cycle problem, which is also NP-complete.

Register Allocation problem

Instance:

1. an undirected unweighted graph G with:
   - vertices representing program variables
   - edges representing pairs of variables which are both needed at the same step of program execution;
2. the number of registers k of the processor.

Problem: is it possible to assign variables to registers so that no edge has both vertices assigned to the same register?

In graph theoretic terms: is it possible to colour the vertices of a graph G with at most k colours so that no edge has both end points of the same colour? (Graph Colouring problem)

Vertex Cover problem

Instance:

1. an undirected unweighted graph G with vertices and edges;
2. a number k.

Task: is it possible to choose k vertices so that every edge is incident to at least one of the chosen vertices?

Set Cover problem

Instance:

1. a number of items n;
2. a set of bundles where each bundle contains a subset of the items, and
3. a number k.

Task: it it possible to choose k bundles which together contain all n items?

Any generalisation of an NP-hard problem remains NP-hard.
For example, this problem can be generalised by assigning a price to each bundle, and restricting the cost of the chosen bundles to be within a budget b.
The generalisation is NP-complete.

NP-complete problems

- Many other practically important problems are also NP-complete.

- Be careful though: sometimes the distinction between a problem in **P** and an NP-complete problem can be subtle!

Examples in **P** vs NP-complete problems

Problems in **P**:

- Given a graph G and two vertices s and t, is there a simple path from s to t of length <u>at most</u> k?
- Given a CNF formula F such that every clause has at most <u>two</u> variables, does F have a satisfying assignment? (2SAT)
- Given a graph G, does G have a tour where every <u>edge</u> is traversed exactly once? (Euler tour)

NP-complete problems:

- Given a graph G and two vertices s and t, is there a simple path from s to t of length <u>at least</u> k? Is there one of length $|V(G)| - 1$? (Hamiltonian path)
- Given a CNF formula F such that every clause has at most <u>three</u> variables, does F have a satisfying assignment? (3SAT)
- Given a graph G, does G have a tour where every <u>vertex</u> is visited exactly once? (Hamiltonian cycle)

Proving NP-completeness

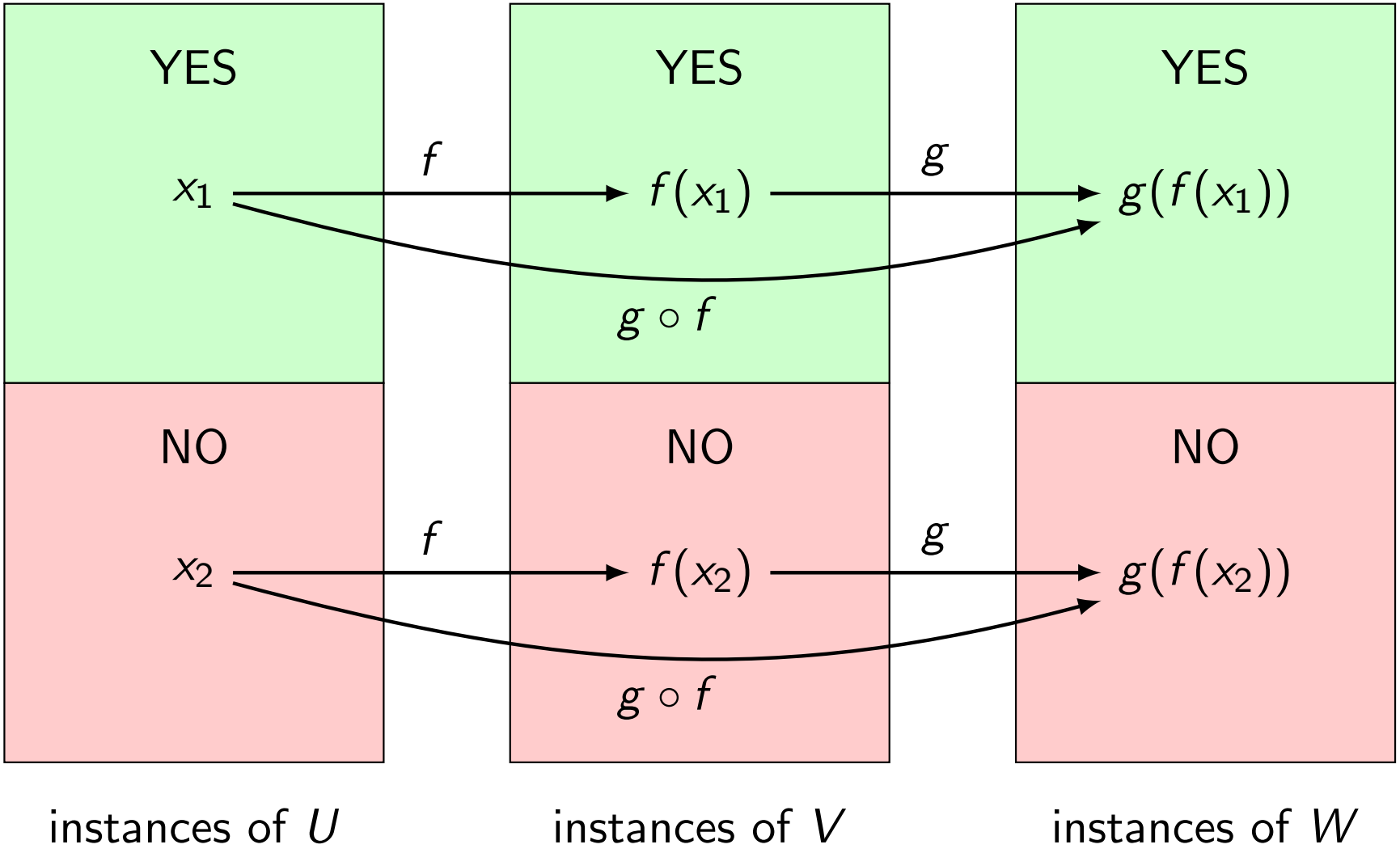
Taking for granted that SAT is NP-complete, how do we prove NP-completeness of another problem in NP?

Theorem

Let V be an NP-hard problem, and let W be another decision problem. If V is polynomially reducible to W, then W is also NP-hard. If, additionally, $W \in \mathbf{NP}$, then W is NP-complete.

Proof Outline

- Let $g(x)$ be a polynomial reduction of V to W, and let U be any problem in NP.
- Since V is NP-hard, there is a polynomial reduction $f(x)$ from U to V.
- Then $(g \circ f)(x)$ is a polynomial reduction from U to W.


YES
YES
YES
x_1
f
$f(x_1)$
g
$g(f(x_1))$
$g \circ f$
NO
NO
NO
x_2
f
$f(x_2)$
g
$g(f(x_2))$
$g \circ f$
instances of U
instances of V
instances of W

Proof

We first claim that $(g \circ f)(x)$ is a reduction from U to W.

- Since f is a reduction from U to V, $U(x)$ is YES iff $V(f(x))$ is YES.
- Since g is a reduction from V to W, $V(f(x))$ is YES iff $W(g(f(x)))$ is YES.

Thus, $U(x)$ is YES iff $W(g(f(x)))$ is YES, i.e., $(g \circ f)(x)$ is a reduction from U to W.

Proof (continued)

- Since $f(x)$ is computed in polynomial time, the length $|f(x)|$ of the output $f(x)$ can be at most a polynomial in $|x|$, i.e., there is a polynomial P such that $|f(x)| \leq P(|x|)$.
- Since $g(y)$ is computed in polynomial time, there is a polynomial Q such that for every input y, the computation of $g(y)$ terminates after at most $Q(|y|)$ steps.
- Thus, the computation of $(g \circ f)(x)$ terminates in at most:
  - $P(|x|)$ steps, for the computation of $f(x)$, plus
  - $Q(|f(x)|) \leq Q(P(|x|))$ steps, for the computation of $g(y)$ (where $y = f(x)$).
- In total, the computation of $(g \circ f)(x)$ terminates in at most $P(|x|) + Q(P(|x|))$ steps, which is polynomial in $|x|$.

Proof (continued)

- Therefore $(g \circ f)(x)$ is a polynomial reduction from U to W.
- But U could be any problem in NP!
- We have now proven that any problem in NP is polynomially reducible to W. Thus, W is NP-hard.
- If W is also in NP, we obtain that W is NP-complete.

- **Sidenote:** W can only be in **NP** if $V \in$ **NP**. If we want to show that W is NP-complete, we need to choose a problem V that we know is NP-complete and reduce it to W.

Example: Reducing 3SAT to VC

Problem

Prove that Vertex Cover (VC) is NP-hard by finding a polynomial time reduction from 3SAT to VC.

Outline

We will map each instance F of 3SAT to a corresponding instance $f(F) = (G, k)$ of VC in polynomial time, and prove that:

1. if F is a YES instance of 3SAT, then $f(F)$ is a YES instance of VC, and
2. if $f(F)$ is a YES instance of VC, then F is a YES instance of 3SAT.

Construction

Given an instance F of 3SAT, construct an instance (G, k) for VC by starting from an empty graph and:

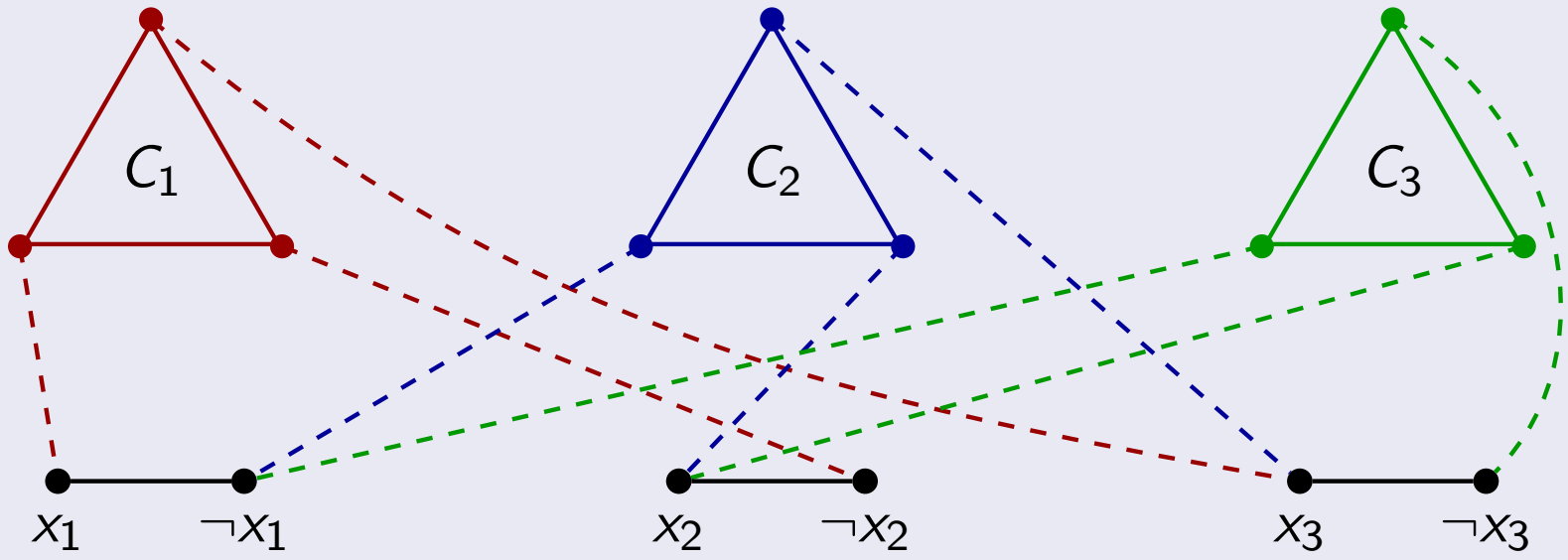
1. for each propositional variable x_i add two new adjacent "literal" vertices x_i and $\neg x_i$;
2. for each clause C_j, add a triangle with three vertices v_1^j, v_2^j, v_3^j and three edges connecting these vertices; each of these vertices represents the occurrence of a literal in the clause;
3. for each clause vertex v_ℓ^j, let x_i (resp., $\neg x_i$) denote the literal it represents, and add an edge between v_ℓ^i and x_i (resp., $\neg x_i$).
4. Finally, set $k = 2m + n$, where m is the number of clauses and n is the number of variables in F.

Example

Consider the propositional formula

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

The corresponding instance of Vertex Cover is:



with $k = 9$.

Example: Reducing 3SAT to VC

Claim

An instance of 3SAT consisting of m clauses and n propositional variables is satisfiable if and only if the corresponding graph has a vertex cover of size at most $k = 2m + n$.

Proof

Assume there is a vertex cover with at most k vertices chosen. Then

1. each pair of literal vertices must have at least one vertex chosen, and
2. each triangle must have at least two vertices chosen.

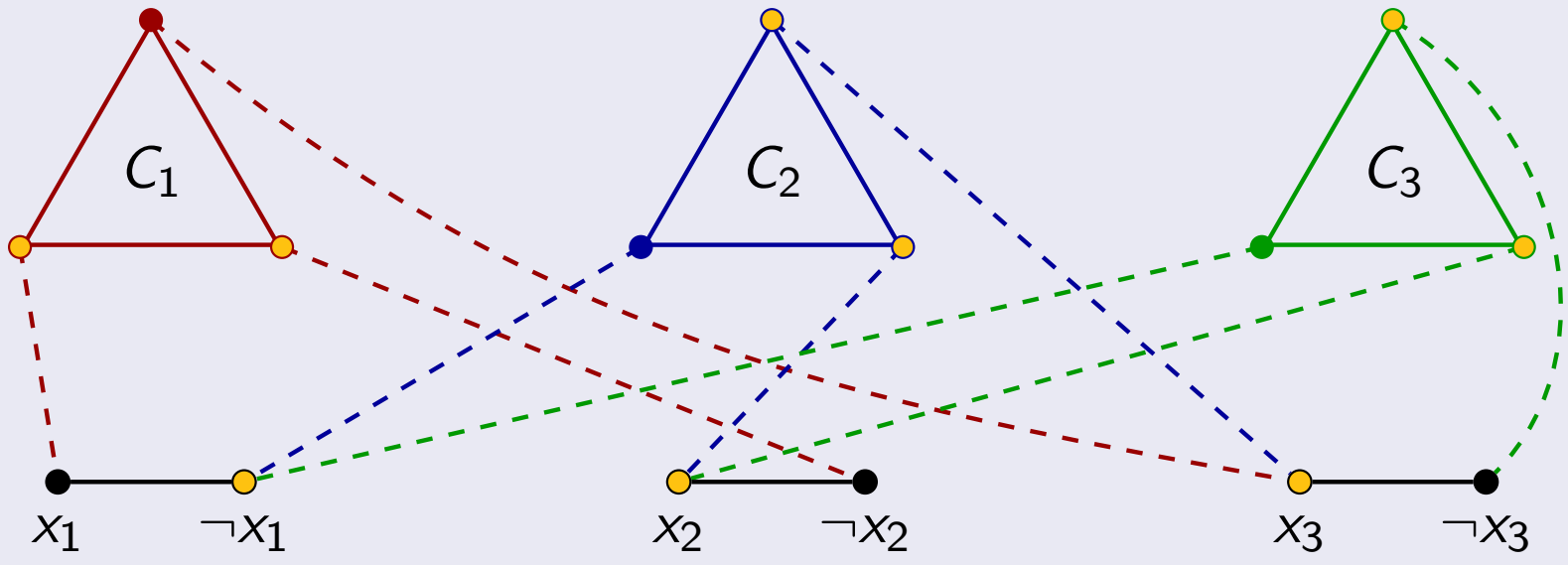
This is in total $2m + n$ vertices; thus each triangle must have <u>exactly</u> two vertices chosen and each pair of literal vertices must have <u>exactly</u> one of its vertices chosen.

Proof (continued)

Recall the example

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

and the corresponding VC instance:



with $k = 9$.

Proof (continued)

- For each variable x_i, set it to true if the vertex x_i is in the vertex cover, and to false if $\neg x_i$ is in the vertex cover.
- Now, in each triangle, for the vertex that is not in the vertex cover, all its neighbours must be in the vertex cover, in particular the neighbour that represents a literal, which is set to true. This guarantees that the clause corresponding to each triangle evaluates to true.

Proof (continued)

- For the reverse direction, assume that the formula has an assignment of the variables which makes it true.
- For each variable x_i, if the assignment sets x_i to true, then add the vertex x_i to the vertex cover, otherwise add the vertex $\neg x_i$ to the vertex cover.
- For each clause, select one of the literals that evaluates to true under the satisfying assignment. Add the two triangle vertices that correspond to the other two literals to the vertex cover.
- In this way we add exactly $2m + n$ vertices to the vertex cover and every edge between a literal vertex and a triangle is covered.

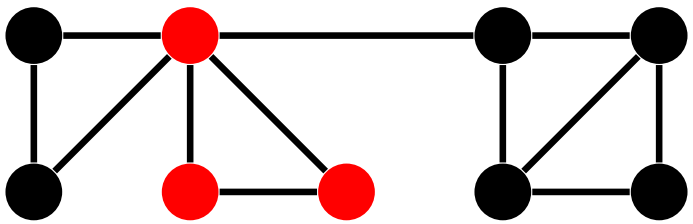
A clique in a graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ such that every two vertices of S are adjacent in G.

Clique Problem

Instance:

- a graph G;
- a positive integer k.

Task: does G have a clique of size k?



Problem

Show that Clique is NP-complete.

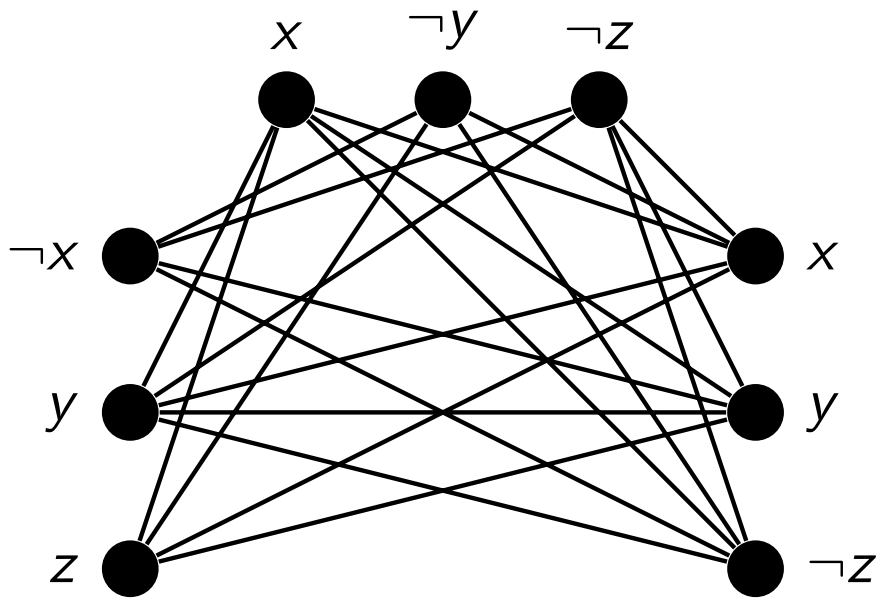
Clique is in NP

- To show that Clique is in NP, we describe what a certificate is and outline a polynomial time algorithm that takes an instance (G, k) of Clique and a certificate C as input and checks whether the certificate is valid.

- A certificate is a subset of vertices $C \subseteq V(G)$ such that $|C| = k$ and every two vertices in C are adjacent in G. The size of this certificate is in $O(|V(G)|)$, so upper bounded by a polynomial in the input size.

- The algorithm checks whether $|C| = k$ and whether every two vertices in C are adjacent in G. This can be done in polynomial time.

$$(\neg x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z) \wedge (x \vee y \vee \neg z)$$



- Let $F = C_1 \wedge C_2 \wedge \ldots C_k$ be a 3-CNF formula
- Construct a graph G that has a clique of size k iff F is satisfiable
- For each clause $C_r = (\ell_1^r \vee \ell_2^r \vee \ell_3^r)$, $1 \leq r \leq k$, add 3 new vertices v_1^r, v_2^r, v_3^r
- Add an edge $\{v_i^r, v_j^s\}$ if

$$\begin{aligned} r &\neq s && \text{and} \\ \ell_i^r &\neq \neg \ell_j^s && \text{where } \neg\neg x = x. \end{aligned}$$

Reducing 3SAT to Clique: Correctness

- F has a satisfying assignment iff G has a clique of size k.
- ($\Rightarrow$): Let α be a satisfying assignment for F. For each clause C_r, choose a literal ℓ_i^r that evaluates to true under assignment α, and select the corresponding vertex s^r in G. Now, the set of selected vertices $\{s^r : 1 \leq r \leq k\}$ is a clique of size k in G since for every variable x, α sets either x to true or it sets $\neg x$ to true, but not both.
- ($\Leftarrow$): Let S be a clique of size k in G. Then, S contains exactly one vertex $s_r \in \{v_1^r, \ldots, v_w^r\}$ for each $r \in \{1, \ldots, k\}$. Denote by ℓ^r the corresponding literal. Now, for any r, r', it is not the case that $\ell_r = \neg \ell_{r'}$ due to the way the edges of G were constructed. Therefore, there is an assignment α such that $\alpha(\ell_r) =$ true for each $r \in \{1, \ldots, k\}$ and α satisfies F.

Reducing 3SAT to Clique: Running time

- The construction can be done in $O(k^2)$ time, which is polynomial in the instance size.

Alternative reduction: VC to Clique

- We could have reduced some other NP-complete problem to Clique in order to prove that Clique is NP-hard.

Definition

The complement of a graph $G = (V, E)$ is the graph $\overline{G} = (V, \overline{E})$ where $\overline{E} = \{\{u, v\} : u, v \in V \text{ and } \{u, v\} \notin E\}$.

Exercise

Show that the following algorithm is a polynomial reduction from VC to Clique.
For an input instance (G, k), the algorithm outputs $(\overline{G}, n - k)$, where n denotes the number of vertices of G.

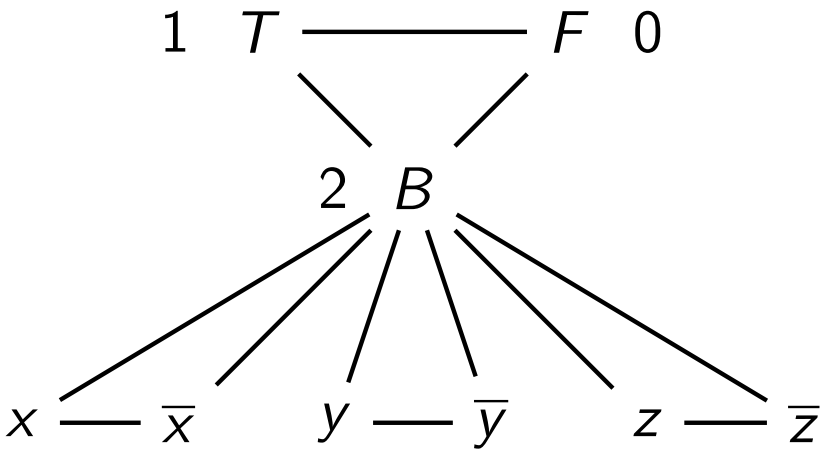
3-Coloring

Instance: an undirected graph $G = (V, E)$
Question: is it possible to assign a colour among $\{0, 1, 2\}$ to each vertex so that no two neighbours receive the same colour?

- The problem is in NP; a certificate is a colouring c of the vertices with up to 3 colours.

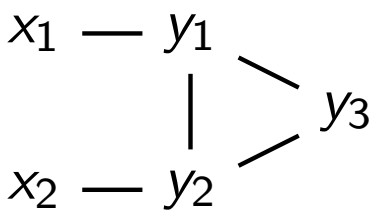
- To show NP-hardness, we'll reduce from 3-SAT

- We use two colours to represent True and False. Wlog., these are 1 for True and 0 for False.
- To make sure that a literal and its negation receive the values 0 and 1, we add a triangle (T, F, B) and literal gadgets that form a triangle with B.



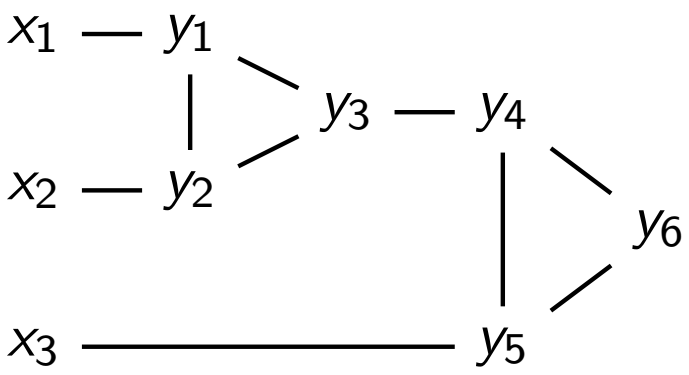
- We'll add a clause gadget for each clause. For a valid colouring of the literal vertices, there should exist a valid colouring for the clause gadget iff the corresponding clause is satisfied by this variable assignment.

An OR-gadget



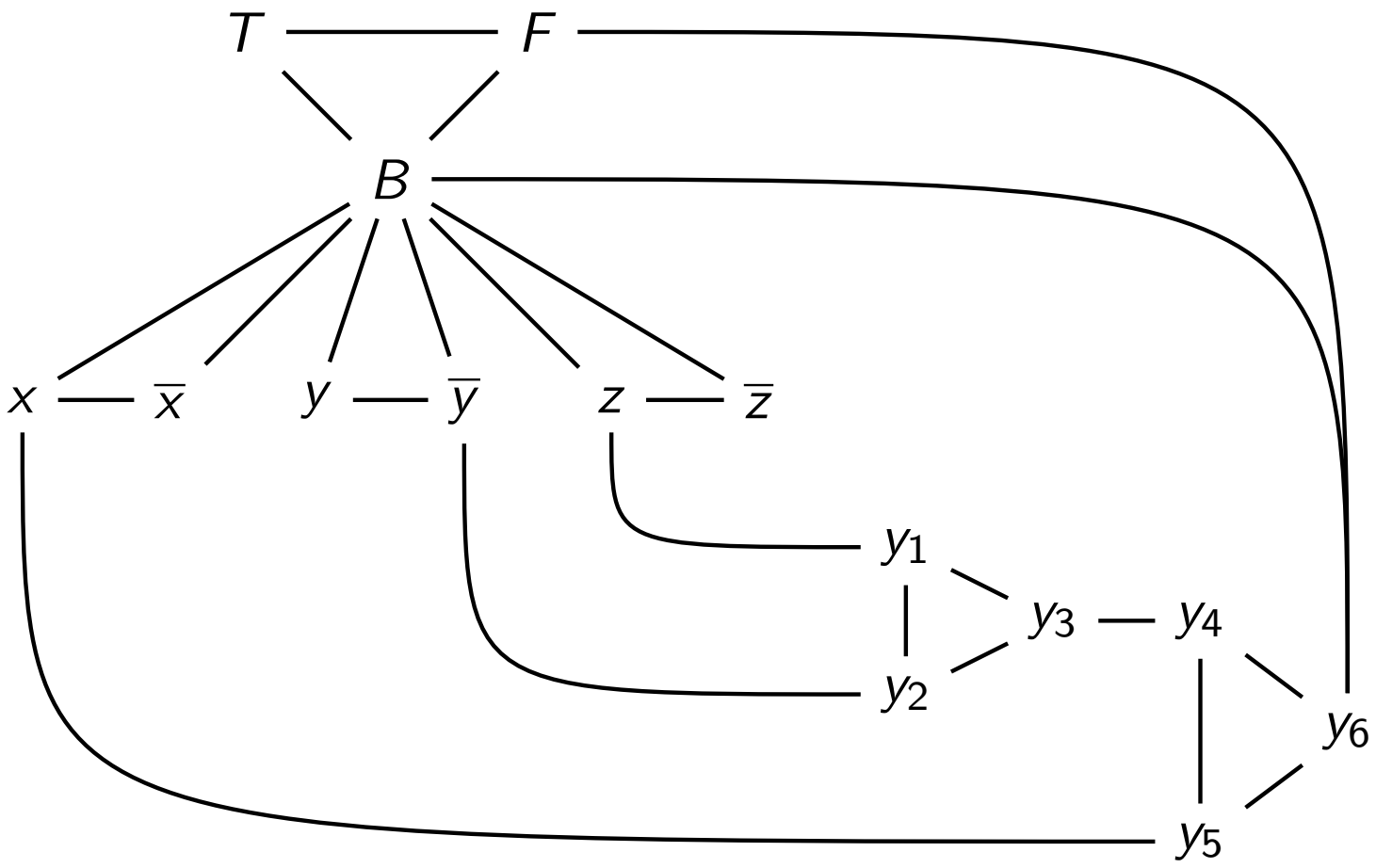
- if $col(x_1) = col(x_2) = 0$, then $col(y_3) = 0$
- if $col(x_1) = 1$ or $col(x_2) = 1$, then it is possible to set $col(y_3) = 1$

A clause gadget



- the clause gadget consists of two OR-gadgets whose inputs (x_1, x_2, x_3) are the literals of the clause, and the output (y_6) is connected to vertices B and F, so as to make the colour 1 the only available option for the output
- this then enures that at least one vertex among x_1, x_2, x_3 is not coloured 0

Example: the clause $(x \vee \overline{y} \vee z)$ is encoded as follows



You should now complete the reduction and prove correctness

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Definition

An optimisation problem is characterised by

- a set of input instances
- a set of feasible solutions for each input instance
- a value for each feasible solution

In a maximisation problem (resp., a minimisation) problem, the goal is to find a feasible solution with maximum (resp., minimum) value.

Example: In the Vertex Cover Minimisation problem, the input is a graph G, the feasible solutions are all the vertex covers of G, and the value of a vertex cover is its size.

Optimisation vs Decision

- Not all decision problems have corresponding optimisation problems. E.g., Satisfiability, Tautology, Reachability
- But many decision problems intrinsically need us to minimise or maximise some quantity.
- **Applications.** Suppose you reduce your problem to Satisfiability in order to use a SAT solver, but the SAT solver finds that your instance is not satisfiable. You could use a MaxSAT solver that maximises the number of satisfied clauses, but a Boolean assignment that leaves some clauses unsatisfied may not make sense in your application. In practice, MaxSAT solvers often distinguish between hard and soft clauses, and the objective is to find an assignment that satisfies all hard clauses and a maximum number of soft clauses.

Optimisation vs Decision

- Suppose we have a decision version and an optimisation version of a problem. Which one is harder to solve?
- Example:
  - decision problem: "Is there a path of length $\leq L$?"
  - optimisation problem: "What is the length of the shortest path?"
- Optimisation problems are no easier than the corresponding decision problems.
- Optimisation problems can usually be solved within a polynomial factor of the corresponding decision problem.

Example: Travelling Salesperson Problem

Travelling Salesperson Problem (Optimisation)

Instance:

- a weighted graph with:
  - vertices representing locations
  - edges representing roads between pairs of locations
  - edge weights representing the lengths of these roads;

Task: find a shortest tour along the edges which visits each location (i.e., vertex) exactly once and returns to the starting location.

Think of a courier who has to deliver mail to several addresses and then return to the post office. How can they do it while travelling the minimum total distance?

Intractable optimisation problems

- The Travelling Salesperson Optimisation Problem is no easier than the corresponding decision problem.
- If we could solve the optimisation version, we could then solve the decision version in constant extra time by comparing the length of the shortest path to the length L being tested.
- We don't know of any polynomial time algorithm for TSP (Decision), so we certainly don't know of any polynomial time algorithm for TSP (Optimisation).

Problem types and complexity classes

- Decision problems: YES/NO answer, depending on whether a certificate exists
  Complexity classes: **P**, **NP**, **coNP**, **PSPACE**, **EXPTIME**

- Optimisation problems: optimal value of a certificate
  Complexity classes: **NPO**, **APX**

- Counting problem: number of certificates
  Complexity classes: **#P**

- Function problem: arbitrary string
  Complexity classes: **FP**, **FNP**

Complexity Zoo: https://complexityzoo.net

The significance of intractable optimisation problems

- It is important to be able to figure out if a problem at hand is intractable in order to know that one has to abandon trying to come up with a polynomial time algorithm.

- So what do we do when we encounter an intractable problem?

Coping with NP-hardness

- **Approximation algorithms.** A vertex cover of G of size at most $2 \cdot \mathrm{OPT}$ can be computed in polynomial time, where OPT is the size of a smallest vertex cover of G.
- **Exact superpolynomial time algorithms.** Vertex Cover can be solved in $O(1.1996^n)$ time [XiaoN17].
- **Fixed parameter algorithms.** Vertex Cover can be solved in $O(1.25284^k + kn)$ time [HarrisN24].
- **Heuristics.** The COVER heuristic (COVer Edges Randomly) finds a smaller vertex cover than state-of-the-art heuristics on a suite of hard benchmark instances [RichterHG07].
- **Restricted inputs.** Vertex Cover can be solved in polynomial time on bipartite, interval graphs, trees, ... [Golumbic04].
- **Quantum algorithms** are not believed to solve NP-hard problems in polynomial time [Aaronson05], but speed-ups are possible. There is a quantum algorithm solving Vertex Cover in $O(1.0953^n)$ time [AmbainisK17, Furer08].

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Problem

Instance: You want to make a salad of lettuce, tomato and onion. You know that:

- 100g of lettuce costs \$1.00, and gives 6 points of texture,
- 100g of tomato costs \$0.80, and gives 4 points of flavour, and
- 100g of onion costs \$0.60, and gives 1 point of flavour and 2 points of texture.

Task: Find an amount of each vegetable so that your salad has:

- weight exactly 300g,
- flavour at least 7, and
- texture at least 7,

at the cheapest possible cost.

Salad

- If we take x_L, x_T and x_O times 100g of each ingredient, we need
  - weight $x_L + x_T + x_O = 3$
  - flavour $4x_T + x_O \geq 7$, and
  - texture $6x_L + 2x_O \geq 7$.
- Subject to these restrictions, we want to minimise the cost $1\,x_L + 0.8\,x_T + 0.6\,x_O$.
- It turns out that the best salad has 110g lettuce, 170g tomato and 20g onion, but we will concentrate on the structure of such problems more than the process of solving them.

Linear Programming

- The restrictions (constraints) and cost (objective function) deal only with linear functions of the variables.
- This is an example of linear programming!
  - Linear programming problems are everywhere in industry.
  - Skills: recognise LP, formulate problem as LP.
- There are polynomial time algorithms for linear programming, such as the ellipsoid algorithm.
- The most popular algorithm for linear programming is the SIMPLEX algorithm, which is worst case exponential but has good average case performance.

- Well optimised solvers available on the Web and in libraries for various programming languages.

- The variables are typically assumed to be non-negative.[2]

[2] An unconstrained variable x can be simulated by the difference between two non-negative variables x' and x^*.

- $\geq$ constraints can be negated to make $\leq$ constraints, e.g.

$$4x_T + x_O \geq 7 \quad \rightarrow \quad -4x_T - x_O \leq -7.$$

- Minimisation problems can be converted to maximisation by negating the objective function, e.g.

$$\min\left[1\,x_L + 0.8\,x_T + 0.6\,x_O\right] \rightarrow \max\left[-1\,x_L - 0.8\,x_T - 0.6\,x_O\right].$$

Salad II

- Suppose now that the vegetables come in pre-portioned 100g packets, and to avoid waste you have decided to use the entire contents of any packet that you open.
- How does this change the problem?
- Now the variables x_L, x_T and x_O are <u>integer</u>-valued rather than real-valued!
- This is an <u>integer</u> linear programming problem.

Question

Does the change from real-valued variables to integer-valued variables make the problem easier or harder (or neither)?

- It might seem to make the problem easier, as the search space now consists of only the points with integer coordinates, not all the points in between.
- However, deciding whether an integer linear program has a feasible solution is NP-complete. This can be proved by reduction from Vertex Cover, for example.

- We have already seen an example of integer linear programming in the integer knapsack problem.
- The variables are the number of times each item is taken.
- There is one inequality constraint arising from the knapsack capacity.
- The objective is to maximise the total value.

Exercise

What additional constraints must be included to reduce 0-1 Knapsack to Integer Linear Programming?

In the **standard form** the objective to be maximised is given by

$$\sum_{i=1}^{n} c_i \, x_i$$

and the constraints are of the form

$$\sum_{i=1}^{n} a_{ij} x_i \leq b_j \qquad (1 \leq j \leq m);$$

$$x_i \geq 0 \qquad (1 \leq i \leq n).$$

- To get a more compact representation of linear programs, we use vectors and matrices.

- Let $\mathbf{x}$ represent a (column) vector,

$$\mathbf{x} = \langle x_1 \dots x_n \rangle^T.$$

- Define a partial ordering on the vectors in $\mathbb{R}^n$ by $\mathbf{x} \leq \mathbf{y}$ if and only if the corresponding inequalities hold coordinate-wise, i.e., if and only if $x_i \leq y_i$ for all $1 \leq i \leq n$.

Write the coefficients in the objective function as

$$\mathbf{c} = \langle c_1 \dots c_n \rangle^T \in \mathbb{R}^n,$$

the coefficients in the constraints as an $m \times n$ matrix

$$A = (a_{ij})$$

and the right-hand side values of the constraints as

$$\mathbf{b} = \langle b_1 \dots b_m \rangle^T \in \mathbb{R}^m.$$

Then the standard form can be formulated simply as:

- maximize $\mathbf{c}^T\mathbf{x}$

- subject to the following two (matrix-vector) constraints:

$$A\mathbf{x} \leq \mathbf{b}$$
$$\mathbf{x} \geq \mathbf{0}.$$

Thus, a Linear Programming optimisation problem can be specified as a triplet $(A, \mathbf{b}, \mathbf{c})$, which is the form accepted by most standard LP solvers.

Translating other constraints to Standard Form

- The Standard Form doesn't immediately appear to handle the full generality of LP problems.
- LP problems could have:
  - equality constraints
  - unconstrained variables (i.e., potentially negative values x_i)
  - absolute value constraints

Equality constraints

- An LP problem may include equality constraints of the form

$$\sum_{i=1}^{n} a_{ij} x_i = b_j.$$

- Each of these can be replaced by two inequalities:

$$\sum_{i=1}^{n} a_{ij} x_i \geq b_j$$
$$\sum_{i=1}^{n} a_{ij} x_i \leq b_j.$$

- Thus, we can assume that all constraints are inequalities.

Unconstrained variables

- In general, a "natural formulation" of a problem as a Linear Program does not necessarily require that all variables be non-negative.
- However, the Standard Form does impose this constraint.
- This poses no problem, because each occurrence of an unconstrained variable x_i can be replaced by the expression

$$x_i' - x_i^*$$

where x_i', x_i^* are new variables satisfying the inequality constraints

$$x_i' \geq 0, \; x_i^* \geq 0.$$

Absolute value constraints

- For a vector
$$\mathbf{x} = \langle x_1, \ldots, x_n \rangle^T,$$
we can define
$$|\mathbf{x}| = \langle |x_1|, \ldots, |x_n| \rangle^T.$$
- Some problems are naturally translated into constraints of the form
$$|A\mathbf{x}| \leq \mathbf{b}.$$
- This also poses no problem because we can replace such constraints with two linear constraints:
$$A\mathbf{x} \leq \mathbf{b} \text{ and } -A\mathbf{x} \leq \mathbf{b},$$
because $|x| \leq y$ if and only if $x \leq y$ and $-x \leq y$.

Summary of Standard Form

- Standard Form: maximize

$$\mathbf{c}^T\mathbf{x}$$

subject to

$$A\mathbf{x} \leq \mathbf{b}$$

and

$$\mathbf{x} \geq \mathbf{0}.$$

- Any vector $\mathbf{x}$ which satisfies the two constraints is called a <u>feasible solution</u>, regardless of what the corresponding objective value $\mathbf{c}^T\mathbf{x}$ might be.

As an example, let us consider the following optimisation problem.

Problem

$$\begin{aligned}
&\text{maximise} && z(x_1, x_2, x_3) = 3x_1 + x_2 + 2x_3 && (3)\\
&\text{subject to} && && \\
& && x_1 + x_2 + 3x_3 \leq 30 && (4)\\
& && 2x_1 + 2x_2 + 5x_3 \leq 24 && (5)\\
& && 4x_1 + x_2 + 2x_3 \leq 36 && (6)\\
& && x_1, x_2, x_3 \geq 0 && (7)
\end{aligned}$$

Let us upper bound the value of the objective function.

Dual LP: example

How large can the value of the objective

$$z(x_1, x_2, x_3) = 3x_1 + x_2 + 2x_3$$

be, without violating the constraints?
We can achieve a crude upper bound by adding inequalities (4) and (5), to obtain

$$3x_1 + 3x_2 + 8x_3 \leq 54.$$

Since all variables are constrained to be non-negative, we are assured that

$$3x_1 + x_2 + 2x_3 \leq 3x_1 + 3x_2 + 8x_3 \leq 54,$$

i.e., the objective cannot exceed 54. Can we do better?

Adding half of inequality (5) and half of (6), we obtain that

$$3x_1 + 1.5x_2 + 3.5 \cdot x_3 \leq 30.$$

In general, we could add up any fraction y_i of each inequality i.

We could try to look for coefficients $y_1, y_2, y_3 \geq 0$ to be used to form a linear combination of the constraints:

$$y_1(x_1 + x_2 + 3x_3) \leq 30y_1 \tag{8}$$
$$y_2(2x_1 + 2x_2 + 5x_3) \leq 24y_2 \tag{9}$$
$$y_3(4x_1 + x_2 + 2x_3) \leq 36y_3 \tag{10}$$

Then, summing up all these inequalities and factoring, we get

$$\begin{aligned} &x_1(y_1 + 2y_2 + 4y_3) \\ &+ x_2(y_1 + 2y_2 + y_3) \\ &+ x_3(3y_1 + 5y_2 + 2y_3) \\ &\leq 30y_1 + 24y_2 + 36y_3. \end{aligned}$$

If we compare this with our objective (3) we see that if we choose y_1, y_2 and y_3 so that:

$$\begin{aligned} y_1 + 2y_2 + 4y_3 &\geq 3 \\ y_1 + 2y_2 + y_3 &\geq 1 \\ 3y_1 + 5y_2 + 2y_3 &\geq 2 \end{aligned}$$

then

$$\begin{aligned} 3x_1 + x_2 + 2x_3 \leq\ & x_1(y_1 + 2y_2 + 4y_3) \\ & + x_2(y_1 + 2y_2 + y_3) \\ & + x_3(3y_1 + 5y_2 + 2y_3). \end{aligned}$$

Combining this with (8) – (10) we get

$$30y_1 + 24y_2 + 36y_3 \geq 3x_1 + x_2 + 2x_3 = z(x_1, x_2, x_3).$$

Consequently, in order to find a tight upper bound for our objective $z(x_1, x_2, x_3)$ in the original problem P, we have to find y_1, y_2, y_3 which solve problem P^*:

$$\text{minimise:} \quad z^*(y_1, y_2, y_3) = 30y_1 + 24y_2 + 36y_3 \tag{11}$$

subject to:

$$y_1 + 2y_2 + 4y_3 \geq 3 \tag{12}$$

$$y_1 + 2y_2 + y_3 \geq 1 \tag{13}$$

$$3y_1 + 5y_2 + 2y_3 \geq 2 \tag{14}$$

$$y_1, y_2, y_3 \geq 0 \tag{15}$$

Then

$$\begin{aligned} z^*(y_1, y_2, y_3) &= 30y_1 + 24y_2 + 36y_3 \\ &\geq 3x_1 + x_2 + 2x_3 \\ &= z(x_1, x_2, x_3) \end{aligned}$$

will be a tight upper bound.
The new problem P^* is called the <u>dual problem</u> of P.

Let us now repeat the whole procedure in order to find the dual of P^*, which will be denoted $(P^*)^*$.
We are now looking for $z_1, z_2, z_3 \geq 0$ to multiply inequalities (12)–(14) and obtain

$$\begin{aligned} z_1(y_1 + 2y_2 + 4y_3) &\geq 3z_1 \\ z_2(y_1 + 2y_2 + y_3) &\geq z_2 \\ z_3(3y_1 + 5y_2 + 2y_3) &\geq 2z_3 \end{aligned}$$

Summing these up and factoring produces

$$\begin{aligned} & y_1(z_1 + z_2 + 3z_3) \\ & + y_2(2z_1 + 2z_2 + 5z_3) \\ & + y_3(4z_1 + z_2 + 2z_3) \\ & \geq 3z_1 + z_2 + 2z_3 \end{aligned} \tag{16}$$

If we choose multipliers z_1, z_2, z_3 so that

$$\begin{aligned} z_1 + z_2 + 3z_3 &\leq 30 \\ 2z_1 + 2z_2 + 5z_3 &\leq 24 \\ 4z_1 + z_2 + 2z_3 &\leq 36 \end{aligned}$$

we will have:

$$\begin{aligned} & y_1(z_1 + z_2 + 3z_3) \\ & + y_2(2z_1 + 2z_2 + 5z_3) \\ & + y_3(4z_1 + z_2 + 2z_3) \\ & \leq 30y_1 + 24y_2 + 36y_3 \end{aligned}$$

Combining this with (16) we get

$$3z_1 + z_2 + 2z_3 \leq 30y_1 + 24y_2 + 36y_3.$$

Consequently, finding the double dual program $(P^*)^*$ amounts to maximising the objective $3z_1 + z_2 + 2z_3$ subject to the constraints

$$\begin{aligned} z_1 + z_2 + 3z_3 &\leq 30 \\ 2z_1 + 2z_2 + 5z_3 &\leq 24 \\ 4z_1 + z_2 + 2z_3 &\leq 36 \\ z_1, z_2, z_3 &\geq 0 \end{aligned}$$

This is exactly our starting program P, with only the variable names changed! Thus, the double dual program $(P^*)^*$ is just P itself.

In general, the primal Linear Program P and its dual P^* are:

$$
\begin{array}{lll}
P: & \text{maximize} & z(\mathbf{x}) = \sum_{i=1}^{n} c_i x_i, \\
 & \text{subject to} & \sum_{i=1}^{n} a_{ij} x_i \leq b_j \qquad (1 \leq j \leq m) \\
 & \text{and} & x_1, \ldots, x_n \geq 0; \\
P^*: & \text{minimize} & z^*(\mathbf{y}) = \sum_{j=1}^{m} b_j y_j, \\
 & \text{subject to} & \sum_{j=1}^{m} a_{ij} y_j \geq c_i \qquad (1 \leq i \leq n) \\
 & \text{and} & y_1, \ldots, y_m \geq 0.
\end{array}
$$

We can equivalently write P and P^* in matrix form:

$$
\begin{aligned}
P: \quad & \text{maximize} && z(\mathbf{x}) = \mathbf{c}^T\mathbf{x}, \\
& \text{subject to} && A\mathbf{x} \leq \mathbf{b} \\
& \text{and} && \mathbf{x} \geq 0; \\
P^*: \quad & \text{minimize} && z^*(\mathbf{y}) = \mathbf{b}^T\mathbf{y}, \\
& \text{subject to} && A^T\mathbf{y} \geq \mathbf{c} \\
& \text{and} && \mathbf{y} \geq 0.
\end{aligned}
$$

Weak Duality Theorem

Theorem

If $\mathbf{x} = \langle x_1 \dots x_n \rangle$ is any feasible solution for P and $\mathbf{y} = \langle y_1 \dots y_m \rangle$ is any feasible solution for P^*, then:

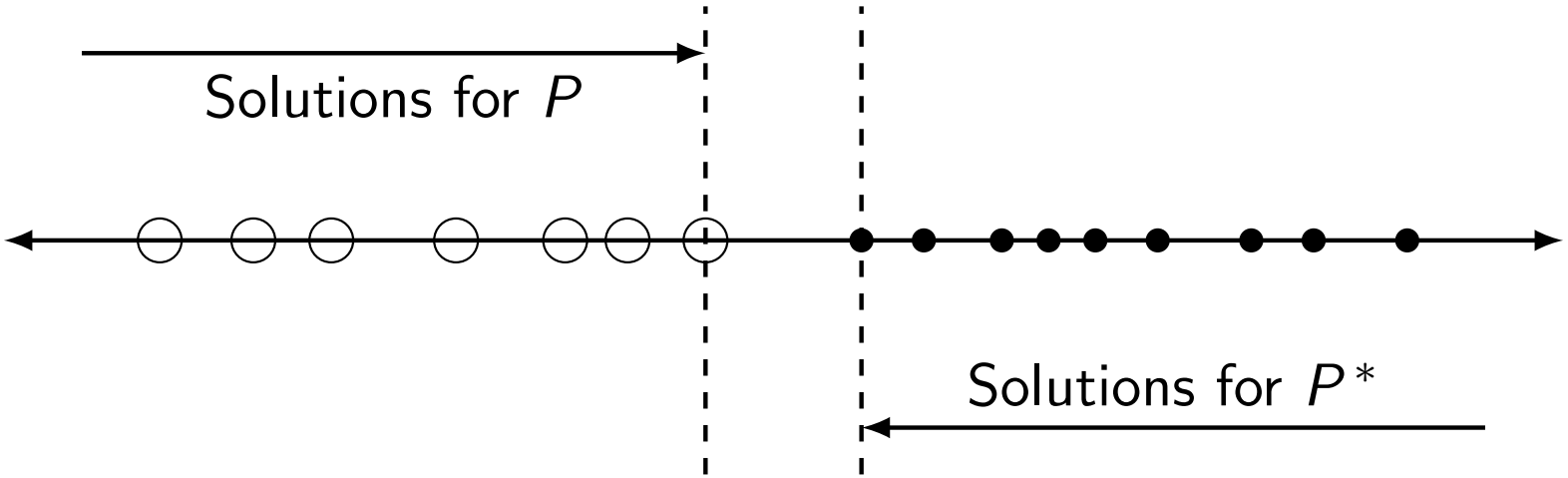
$$z(\mathbf{x}) = \sum_{i=1}^{n} c_i x_i \leq \sum_{j=1}^{m} b_j y_j = z^*(\mathbf{y})$$

Proof

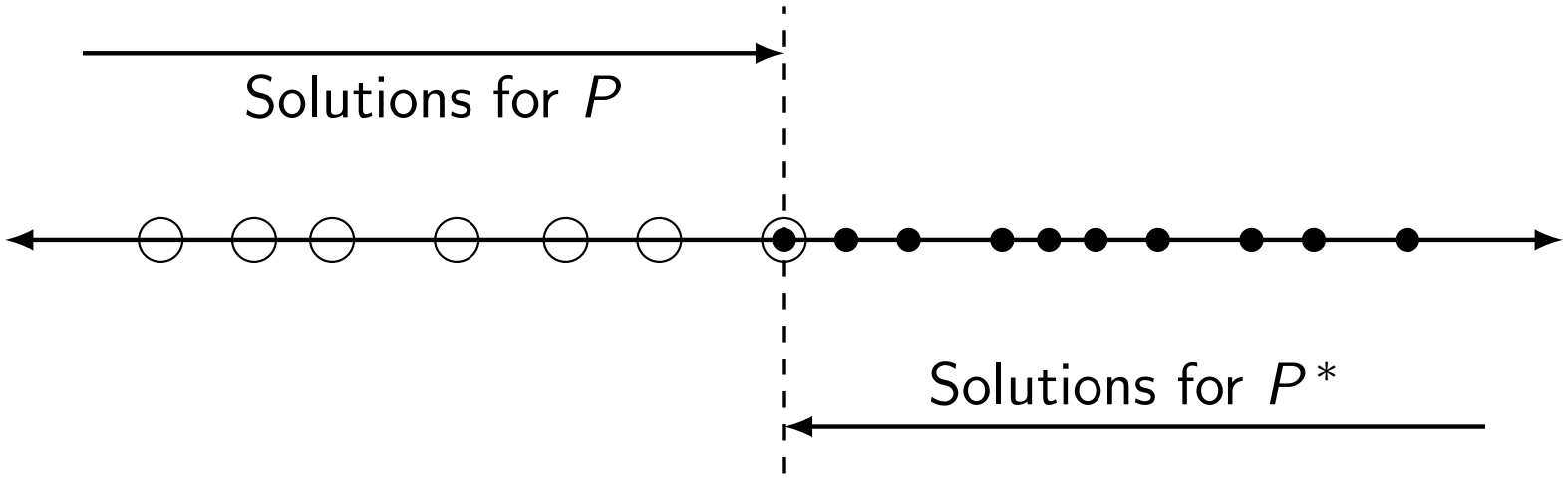
Since $\mathbf{x}$ and $\mathbf{y}$ are feasible solutions for P and P^* respectively, we can use the constraint inequalities, first from P^* and then from P to obtain

$$\begin{aligned} z(\mathbf{x}) &= \sum_{i=1}^{n} c_i x_i \leq \sum_{i=1}^{n} \left(\sum_{j=1}^{m} a_{ij} y_j \right) x_i \\ &= \sum_{j=1}^{m} \left(\sum_{i=1}^{n} a_{ij} x_i \right) y_j \leq \sum_{j=1}^{m} b_j y_j \\ &= z^*(\mathbf{y}). \end{aligned}$$

Thus, the value of (the objective of P^* for) any feasible solution of P^* is an upper bound for the set of all values of (the objective of P for) all feasible solutions of P, and every feasible solution of P is a lower bound for the set of feasible solutions for P^*.



If we find a feasible solution for P which is equal to a feasible solution to P^*, this common value must be the maximum feasible value of the objective of P and the minimum feasible value of the objective of P^*.



Strong Duality Theorem

Theorem

If $\mathbf{x} = \langle x_1 \dots x_n \rangle$ is an optimal solution for P and $\mathbf{y} = \langle y_1 \dots y_m \rangle$ is an optimal solution for P^*, then:

$$z(\mathbf{x}) = \sum_{i=1}^{n} c_i x_i = \sum_{j=1}^{m} b_j y_j = z^*(\mathbf{y})$$

Strong Duality Theorem

- If we use a search procedure to find an optimal solution for P we know when to stop: when such a value is also a feasible solution for P^*.

- The most commonly used LP solving method, the SIMPLEX method, produces an optimal solution for P: it stops at a value of the primal objective which is also a value of the dual objective.

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Superpolynomial algorithms

- Even if a problem is intractable, we don't necessarily have to abandon our usual techniques for algorithm design.
- In a practical scenario, there are often degrees to the intractability of a problem.
  - The brute force might run for an unreasonably long time for all but the smallest instances, whereas a faster (but still slower than polynomial) algorithm might still be viable for moderately small instances.
- Even if we can't find a polynomial time algorithm, there are sometimes significant benefits to:
  - reducing the base of the exponential term, or
  - changing from factorial time to exponential time.

.

Problem

Instance: an array of n positive integers, and a positive integer S.
Task: find the maximum sum of a subarray (not necessarily consecutive) with sum $\leq S$.

Example

If the array is $[2, 5, 6, 8]$ and the target is 12, then the answer is 11 (from $5 + 6$).

- A brute force algorithm for this problem would try every possible subset.
- There are two choices for each number (include it or exclude it in the subset), for a total of 2^n subsets.
- We then have to add up each subset, for a total time complexity of $O(2^n\, n)$.

Question

Can we do better? Smaller exponent perhaps?

- Some redundancy in the brute-force algorithm!
  - Each set of choices from the first half of the array is represented in $2^{n/2}$ different subsets.
  - After a change in the first half, we will again go through $2^{n/2}$ corresponding subsets for the various selections from the second half.
  - Solve each half separately?
- We can try adding up every subset of the first half, and separately every subset of the second half, and look for a pair which matches.
  - Given two arrays, find a pair which adds to as close to S as possible.
- This technique is called <u>meet in the middle</u>.

Solution

- Compute all the sums of subsets of the first half and the second half of the array by brute force, and store them in arrays A and B.
- Sort array B using mergesort.
- For every entry of array A, say $A[i]$, find the best match in array B. This is the largest entry $B[j]$ so that $B[j] \leq S - A[i]$. We find it using binary search.
- Update the tentative answer to $A[i] + B[j]$ if it is larger than what we found so far.

- Clearly correct; every subset of the array combines a subset of each half.
- Time complexity:
  - Computing arrays A and B takes $O(2^{n/2}\, n)$ time.
  - **Note:** in the word-RAM model of computation, we may need two words to represent the entries of A and B.
  - Array B has $2^{n/2}$ entries, so mergesort runs in $O(2^{n/2}\, n)$ time.
  - For each of $2^{n/2}$ entries of A, we run a binary search in $O(\log 2^{n/2}) = O(n)$ time.
- Therefore the total time complexity is $O(2^{n/2}\, n)$.

- Our new complexity $O(2^{n/2}\, n)$ is still exponential, but about the square root of the brute force complexity $O(2^n n)$.
- For example, assuming 10^9 operations per second (and ignoring constant factors),
  - the brute force will take more than <u>2,000 years</u> to solve an instance with $n = 60$, whereas
  - our new algorithm will take around a minute!
- Trade-off: we need to store all the sums from each half of the array, using $O(2^{n/2})$ memory instead of the brute force's $O(n)$.
- Schroeppel and Shamir (1979) designed an algorithm using $O^*(2^{n/2})$ time and $O^*(2^{n/4})$ space. Here, the O^*-notation ignores polynomial factors.

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Definition

An approximation algorithm A for an optimisation problem Π is a polynomial time algorithm that returns a feasible solution.
Denote by $A(I)$ the value of the feasible solution returned by the approximation algorithm A for an instance I and by $\mathrm{OPT}(I)$ the value of the optimum solution.
If Π is a minimisation problem, the approximation ratio of A is r if

$$\frac{A(I)}{\mathrm{OPT}(I)} \leq r \text{ for every instance } I.$$

If Π is a maximisation problem, the approximation ratio of A is r if

$$\frac{\mathrm{OPT}(I)}{A(I)} \leq r \text{ for every instance } I.$$

A is an r-approximation algorithm if it has approximation ratio r.

Minimum Vertex Cover

Problem

Instance: an undirected unweighted graph $G = (V, E)$

Feasible solution: subset of vertices S so that every edge is incident to at least one vertex from S

Objective: minimise size

Approximate Minimum Vertex Cover

Algorithm

- Pick an arbitrary edge and include both of its endpoints in the vertex cover.
- Remove these vertices and all incident edges from the graph, and repeat.
- Continue picking edges in this way until no edges are left.

- This produces a vertex cover, because edges are removed only when they are covered, and we perform this procedure until no edges are left.
- For each edge the algorithm picks, it includes both end points into the vertex cover, and a smallest vertex cover needs to include at least one of them.
- Thus, we have produced a vertex cover of size at most twice the size of the smallest vertex cover.

Problem

Instance: A list of n discrete items described by their weights w_i and values v_i, and a maximal weight limit W of your knapsack.

Task: Find a subset S of items with total weight not exceeding W and maximal total value.

- We saw that greedily taking the item of highest density (value per unit weight) is not optimal.
- Is it a good approximation?

0-1 Knapsack: density heuristic

- Sadly, no. An extreme counterexample is:

$$W = 1000$$
$$w_1 = 1000 \qquad v_1 = 1000$$
$$w_2 = 1 \qquad v_2 = 1.001$$

  Here, adding item 2 prevents us from taking item 1.

- We might miss out on items of large value

- Alternative greedy strategy: pick items of highest value

0-1 Knapsack: value heuristic

- Greedily add the highest value item

- This is very bad precisely when our other heuristic was good:

$$
\begin{aligned}
W &= 1000 \\
w_1 &= 1000 & v_1 &= 1.001 \\
w_2 &= 1 & v_2 &= 1 \\
&\vdots & &\vdots \\
w_{1001} &= 1 & v_{1001} &= 1.
\end{aligned}
$$

0-1 Knapsack Approximation

- Is it possible for <u>both</u> heuristics to fail badly on the <u>same</u> instance?
- Perhaps surprisingly, no!
- At least one of them gets within half of the correct answer.

0-1 Knapsack Approximation

- Let V_{opt} be the value of an optimal solution, and V_{dens} and V_{val} be the totals achieved by greedy selection using the density heuristic and the value heuristic respectively.

- We'll show that
$$V_{dens} + V_{val} > V_{\text{opt}},$$
so
$$\max(V_{dens}, V_{val}) > \frac{1}{2} V_{\text{opt}}.$$

- How? Key observation is that the density heuristic is already pretty good; it just might not leave room for the next item we want to pick.

Lemma

Suppose the greedy algorithm using the density heuristic picks items $I \subseteq \{1, \ldots, n\}$, and that item j is the highest density item not in this selection.

Then

$$\sum_{i \in I} v_i + v_j > V_{\text{opt}}.$$

Proof

First, this quantity is greater than $\sum_{i \in I} v_i$ plus part of v_j (calculated by the remaining weight capacity), which is a feasible solution to the corresponding instance of fractional knapsack.

But any selection for 0-1 knapsack is also a valid selection for fractional knapsack, so this is an upper bound for the solution to 0-1 knapsack.

Therefore

$$\sum_{i \in I} v_i + v_j > \sum_{i \in I} v_i + \left(1 - \frac{\sum_{i \in I} w_i}{W}\right) v_j \geq V_{\text{opt}}.$$

Corollary

$V_{dens} + V_{val} > V_{\text{opt}}$

Proof

$\sum_{i \in I} v_i$ is exactly V_{dens}, and $v_j \leq V_{val}$ because otherwise the value heuristic would pick item j first.

Corollary

$\max(V_{dens}, V_{val}) > \frac{1}{2} V_{\text{opt}}$, as required.

Problem

Instance: a complete weighted graph G with weights $d(i,j)$ of edges (to be interpreted as distances) satisfying the "triangle inequality": for any three vertices i, j, k, we have

$$d(i,k) \leq d(i,j) + d(j,k).$$

Feasible solution: tour visiting all vertices

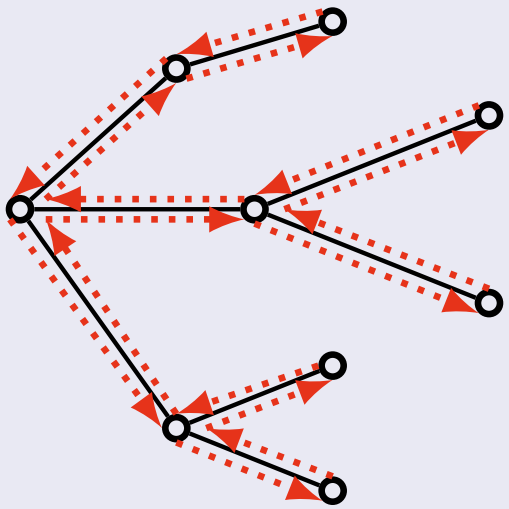
Objective: minimise tour length

Claim

Metric TSP has an approximation algorithm producing a tour of total length at most twice the length of the shortest tour, which we will denote by opt.

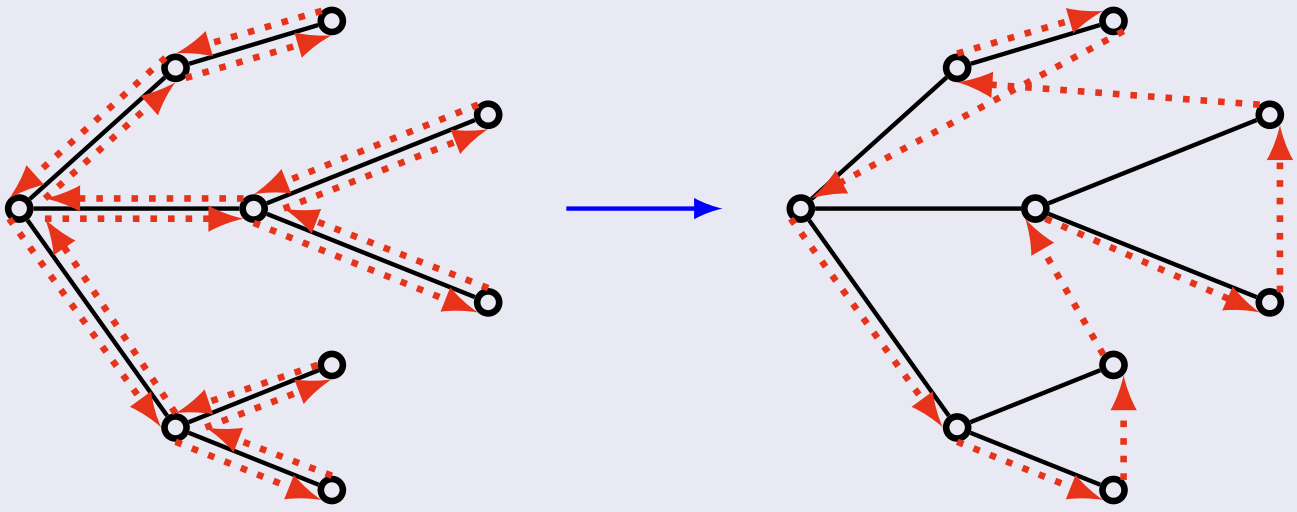
Algorithm + Analysis

Find a minimum spanning tree T of G. Since the optimal tour with one of its edges e removed represents a spanning tree, we have that the total weight of T satisfies $w(T) \leq opt - w(e) \leq opt$. If we do a depth first traversal of the tree, we will travel a total distance of $2w(T) \leq 2opt$.



Algorithm + Analysis (continued)

We now take shortcuts to avoid visiting vertices more than once; because of the triangle inequality, this operation does not increase the length of the tour.



Approximating the general TSP

- Unfortunately, the general Travelling Salesperson Problem does not allow any constant factor approximation algorithm at all: if $\mathbf{P} \neq \mathbf{NP}$, then for no $c > 1$ can there be a polynomial time algorithm which for every instance produces a tour which is at most c times the length of the shortest tour!
- To prove this, we show that if for some $c > 1$ there was indeed a polynomial time algorithm producing a tour which is at most c times the length of the shortest tour, then we could obtain a polynomial time algorithm which solves the Hamiltonian Cycle problem.
- This is the problem of determining for a graph G whether G contains a cycle visiting all vertices exactly once. It is known to be NP-complete.

Approximating the general TSP

- Let G be an arbitrary unweighted graph with $n \geq 2$ vertices.
- We turn this graph into a complete weighted graph G^* by setting the weights of all existing edges to 1, and then adding edges of weight $c \cdot n$ between the remaining pairs of vertices.
- If an approximation algorithm for TSP exists, it produces a tour of all vertices with total length at most $c \cdot opt$, where opt is the length of the optimal tour through G^*.

- If the original graph G has a Hamiltonian cycle, then G^* has a tour consisting of edges already in G and of weights equal to 1, so such a tour has length of exactly n.

- Otherwise, if G does not have a Hamiltonian cycle, then the optimal tour through G^* must contain at least one added edge of length $c \cdot n$, so

$$opt \geq (c \cdot n) + (n - 1) \cdot 1 > c \cdot n.$$

Approximating the general TSP

- Thus, our approximate TSP algorithm either returns:
  - a tour of length at most $c \cdot n$, indicating that G has a Hamiltonian cycle, or
  - a tour of length greater than $c \cdot n$, indicating that G does not have a Hamiltonian cycle.
- If this approximation algorithm runs in polynomial time, we now have a polynomial time decision procedure for determining whether G has a Hamiltonian cycle.
- This can only be the case if $\mathbf{P} = \mathbf{NP}$.

Which coping strategy to choose?

- In some sense, NP-complete problems are equally difficult, because any of them is polynomially reducible to any other.
- However, there are more fine-grained considerations that make drastic differences.
- An algorithmic speed-up from 2^n to $\sqrt{2^n} = 2^{n/2}$ is akin to a speed-up from n^2 to n or from n^4 to n^2.
- For important problems, one would want to implement a variety of coping strategies and run them all.

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

- For a decision problem A, its complement coA is a decision problem, where for each input x, coA(x) = YES iff A(x) = NO

Definition

A problem is in **coNP** (coNP-complete, coNP-hard) iff its complement is in **NP** (NP-complete, NP-hard).

- Examples of coNP-complete problems:
  - Tautology. Given a Boolean formula, is each assignment satisfying?
  - Expander. Given a graph $G = (V, E)$, does each vertex set $S \subseteq V$ with $|S| \leq |V|/2$, have at least $|S|$ vertices in its neighbourhood?
  - Not 3-Colorable. Given a graph $G = (V, E)$, do we need at least 4 colours to properly colour G?
  - Is-Min-Vertex-Cover. Given a graph G and a vertex cover S of G, is S a smallest vertex cover for G?

Definition (short NO-certificates)

A decision problem A is in **coNP** if there is a polynomial-time solvable decision problem B such that

- for every input x, $A(x)$ is NO iff there is some y with $|y| \leq |x|^{O(1)}$ for which $B(x, y)$ is YES.

Definition (all candidate YES-certificates fail~~work~~)

A decision problem A is in **coNP** if there is a polynomial-time solvable decision problem B such that

- for every input x, $A(x)$ is YES iff for each y with $|y| \leq |x|^{O(1)}$ we have that $B(x, y)$ is ~~NO~~.YES.

Note: coP $=$ P.

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

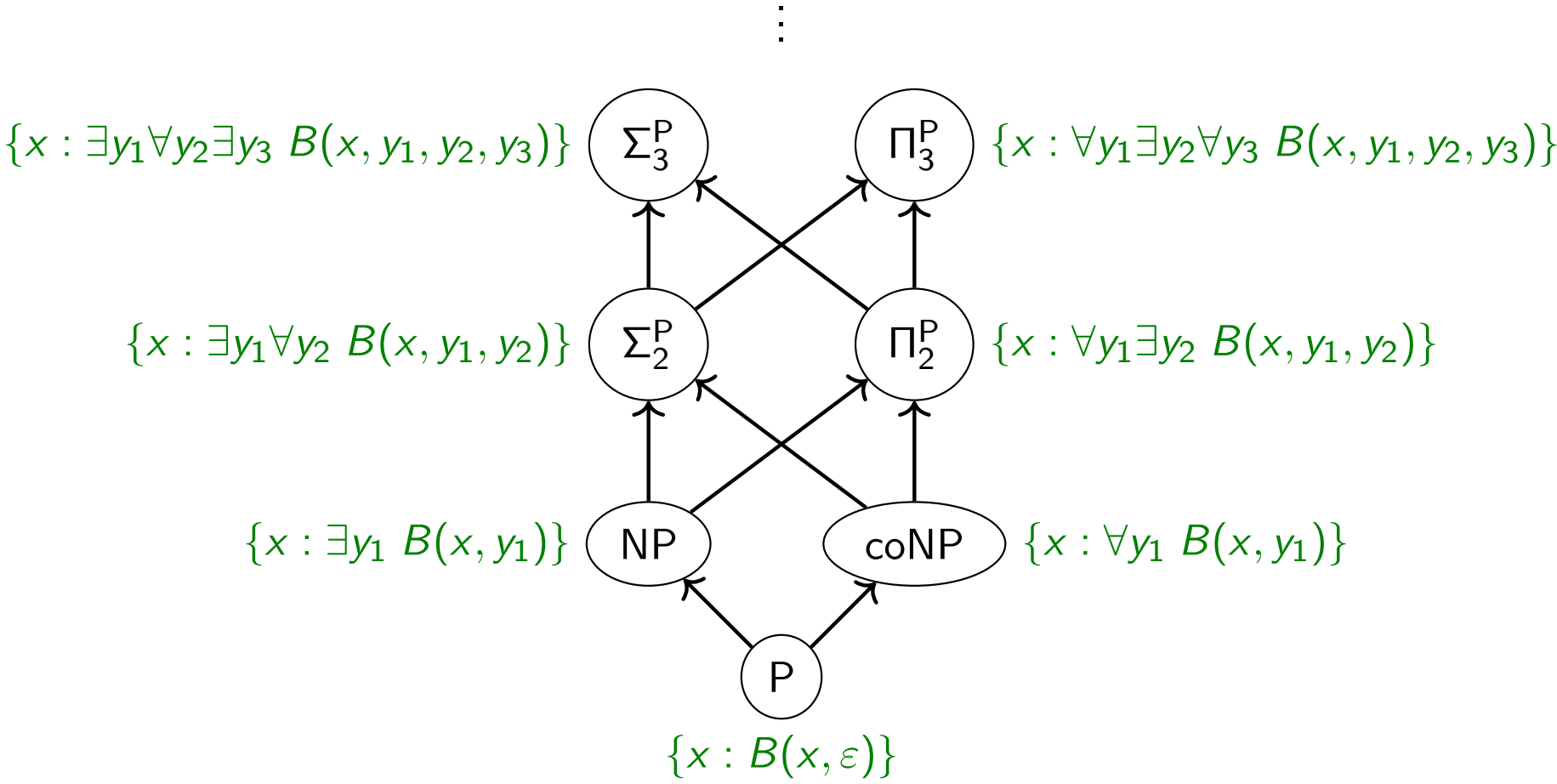
2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

P, NP, coNP, and certificates

- A decision problem can be defined by its set of YES instances
- Suppose B is a poly-time certifier
- problem in P: $\{x : B(x, \varepsilon)\}$, where ε denotes the empty string
- problem in NP: $\{x : \exists y \text{ with } |y| \leq |x|^{O(1)} \text{ , } B(x, y)\}$
- problem in coNP: $\{x : \forall y \text{ with } |y| \leq |x|^{O(1)} \text{ , } B(x, y)\}$


$\vdots$
$\{x : \exists y_1 \forall y_2 \exists y_3\ B(x, y_1, y_2, y_3)\}$
Σ_3^P
Π_3^P
$\{x : \forall y_1 \exists y_2 \forall y_3\ B(x, y_1, y_2, y_3)\}$
$\{x : \exists y_1 \forall y_2\ B(x, y_1, y_2)\}$
Σ_2^P
Π_2^P
$\{x : \forall y_1 \exists y_2\ B(x, y_1, y_2)\}$
$\{x : \exists y_1\ B(x, y_1)\}$
NP
coNP
$\{x : \forall y_1\ B(x, y_1)\}$
P
$\{x : B(x, \varepsilon)\}$

Oracles

- Oracle for a complexity class Π: solves any problem in Π in one computation step
- NP^{Π}: class of languages of the form $\{x : \exists y\ B^{\Pi}(x, y)\}$ where B^{Π} is a polynomial-time certifier that has access to an oracle for Π
- coNP^{Π}: class of languages of the form $\{x : \forall y\ B^{\Pi}(x, y)\}$
- The polynomial hierarchy contains Σ_k^P and Π_k^P for each $k \geq 1$, with

$$\Sigma_k^P = \mathrm{NP}^{\Sigma_{k-1}^P} \qquad \Pi_k^P = \mathrm{coNP}^{\Pi_{k-1}^P} \qquad \text{for } k \geq 2$$

$$\Sigma_1^P = \mathrm{NP} \qquad \Pi_1^P = \mathrm{coNP}$$

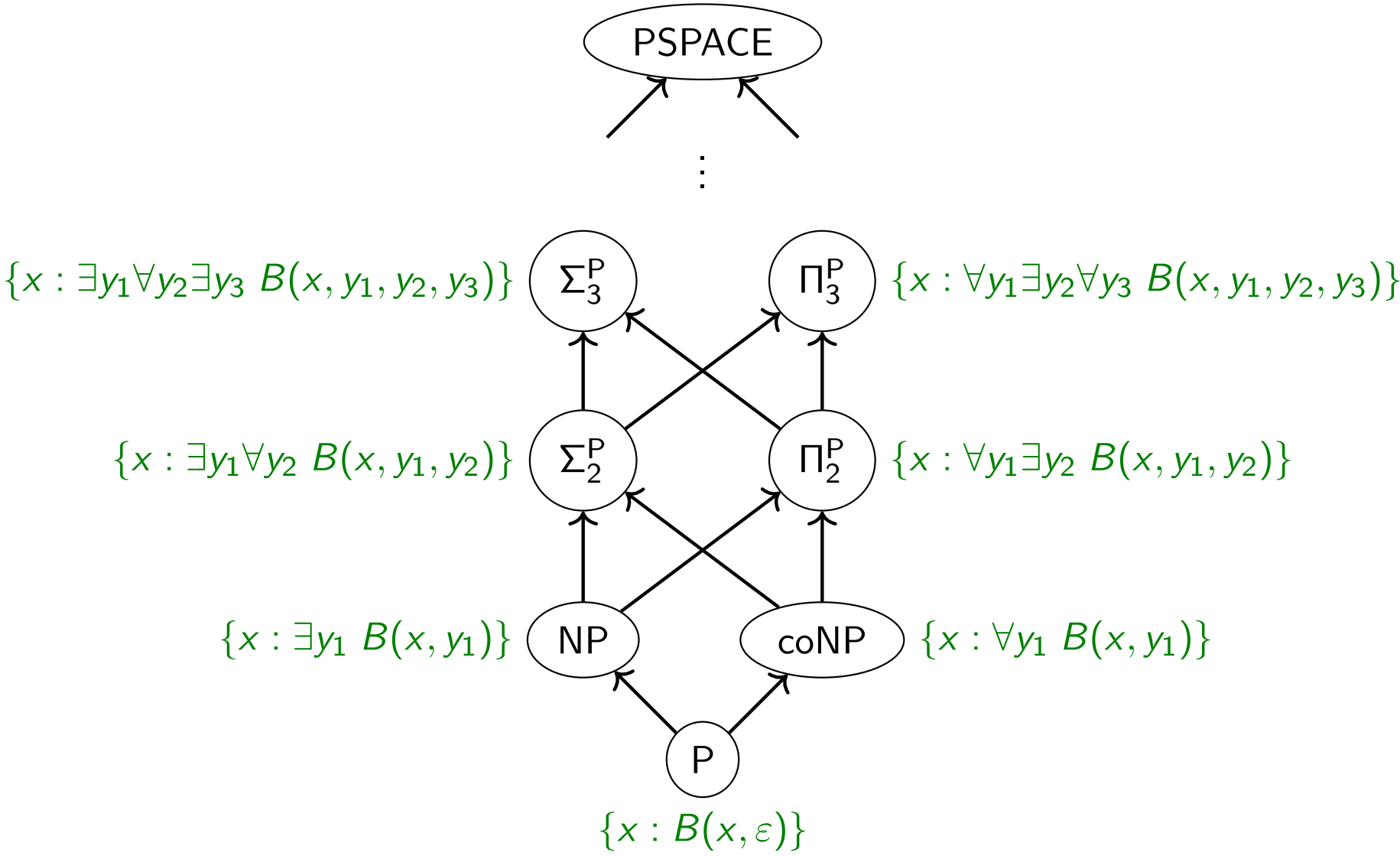

PSPACE
Σ_3^P
Π_3^P
$\{x : \exists y_1 \forall y_2 \exists y_3\ B(x, y_1, y_2, y_3)\}$
$\{x : \forall y_1 \exists y_2 \forall y_3\ B(x, y_1, y_2, y_3)\}$
Σ_2^P
Π_2^P
$\{x : \exists y_1 \forall y_2\ B(x, y_1, y_2)\}$
$\{x : \forall y_1 \exists y_2\ B(x, y_1, y_2)\}$
NP
coNP
$\{x : \exists y_1\ B(x, y_1)\}$
$\{x : \forall y_1\ B(x, y_1)\}$
P
$\{x : B(x, \varepsilon)\}$

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Counting Problems

- Counting problems require to output the number of certificates?
- If a counting problem can be solved in polynomial time, we say it is in **FP**.
- **FP**: class of polynomial-time solvable function problems
- **#P** is the analog of **NP** for counting problems
- A counting problem A is in **#P** if there exists a polynomial-time computable function B and a polynomial poly such that $A(x) = |\{y : B(x, y) \text{ and } |y| \leq \text{poly}(|x|)\}|$.

#P-completeness

- Cook reduction (or polynomial-time Turing reduction): $A \leq_T^P B$ if there is an algorithm that solves A in polynomial time using an oracle for B
- A is **#P**-hard if every problem in **#P** can be Cook reduced to A
- A is **#P**-complete if A is in **#P** and A is **#P**-hard.
- Examples
  - #SAT is **#P**-complete.
  - #Bipartite-Perfect-Matchings is **#P**-complete.

Parsimonious reductions

- Some Karp-reductions preserve the number of certificates
- These are called parsimonious Karp reductions

Exercise

Show that #3SAT is #P-complete.
Hint: Our reduction SAT $\leq_P$ 3SAT is not parsimonious. Can we make it parsimonious?

Table of Contents

1. Decision Problems
1.1 Efficiency of Algorithms
1.2 Polynomial Reductions
1.3 Intractable Decision Problems

2. Optimisation Problems
2.1 Detour: Linear Programming
2.2 Superpolynomial Algorithms
2.3 Approximation Algorithms

3. Beyond NP
3.1 coNP
3.2 Polynomial Hierarchy
3.3 Counting problems and Cook reductions

4. Puzzle

Problem

You are given a coin, but you are not guaranteed that it is a fair coin. It may be biased towards either heads or tails.
Use this coin to simulate a fair coin.

Hint

Try tossing the biased coin more than once!

That's All, Folks!!